

Homework 4:

Intro. to Virtual and Augmented Reality

Project Refinement

학번: 2025021826

이름: 정경서

목차

1. 과제 개요
2. Scene Refinement 및 공간감 강화
3. Avatar A/B 상호작용 구현
4. LLM 기반 AI 반응 시스템
5. AR Marker 기반 오브젝트 배치
6. Windows Build
7. 결론

1. 과제 개요

본 과제는 기존에 구현했던 VR blind date/ flirting scene을 최종 프로젝트 형태로 개선하는 것을 목표로 한다. 과제의 핵심 요구사항은 크게 세 가지로 정리할 수 있다.

첫 번째는 VR scene에 공간감과 장소감을 강화하는 것이다. 이를 위해 바닥, 벽, landmark, 계절감 있는 object, 익숙한 크기의 object 등을 추가하고, occlusion, motion parallax, shadow 등 심리적 3D cue를 활용해야 한다.

두 번째는 2명의 human character를 scene 안에 배치하고, Avatar A는 사용자가 조작하며, Avatar B는 AI가 제어하도록 만드는 것이다. 사용자는 Avatar A의 시점을 공유할 수 있어야 하며, Avatar A는 keyboard, controller, button command, voice command 등 간단한 방식으로 여러 행동을 수행할 수 있어야 한다.

세 번째는 open-source LLM과 간단한 network communication을 이용하여 Avatar B의 반응을 생성하는 것이다. Avatar B는 단순히 고정된 scripted response만 수행하는 것이 아니라, Avatar A의 행동 정보와 현재 상황을 바탕으로 voice, action, expression, emotion을 생성해야 한다.

본 구현에서는 기존 HW2-3의 blind date scene을 기반으로 하여, Girl avatar를 사용자 제어 Avatar A로 설정하고, Boy avatar를 AI 제어 Avatar B로 설정했다. Avatar A의 head/neck 위치에 camera를 배치하여 사용자가 Avatar A의 시점에서 Boy를 바라보도록 구성했다. 또한 Unity와 Python server, 그리고 Ollama 기반 open-source LLM인 llama3.2:3b를 연결하여 Avatar B의 반응을 AI가 생성하도록 구현했다.

추가로, 본 구현에서는 단순히 animation과 facial expression만 실행한 것이 아니라, 각 interaction 상황에 맞는 **voice clip**도 함께 재생되도록 구성했다. 음성은 이전 HW2-3에서 사용한 방식과 같이 AI 음성 생성 프로그램을 이용해 미리 제작한 뒤, Unity의 AudioSource와 AudioClip으로 연결했다. 따라서 Avatar A와 Avatar B의 인사, 작별, 농담, 웃음, 어색한 반응 등이 animation, facial expression, sound와 함께 실행된다. 이는 사용자가 장면의 상호작용 맥락을 더 명확하게 이해하도록 돕고, AI 기반 반응의 감성적 품질을 보완한다.

과제 관련 Unity 구현은 GitHub(https://github.com/KyungseoJung/Unity_Personal_VR2026_HW1-2_360Scene_JKS)의 [360Scene_Desktop_HW4_Marker](#) 이름의 Scene에서 확인 가능하다.

최종 구현 YouTube영상은 <https://youtu.be/G5y0XiLQ2F8>에서 확인 가능하다

2. Scene Refinement 및 공간감 강화

과제 4에서는 “where cue”를 강화하여 사용자가 VR scene 안에서 장소와 공간을 더 명확히

느끼도록 구현했다. 본 구현에서는 기존 360도 video와 blind date scene에 여러 object를 추가하여, 두 캐릭터만 존재하는 빈 장면이 아니라 하나의 구체적인 데이트 장소처럼 느껴지도록 구성했다.

360도 video sphere 안에 두 명의 avatar가 마주 보는 blind date 상황에서 **table cloth, foods, cutlery, vase, cup, camera object, menu board, cherry blossom tree** 등을 추가했다. 특히 table과 table 위의 object들은 Avatar A와 Avatar B 사이에 위치하도록 배치했다. 이를 통해 사용자는 두 캐릭터가 같은 공간 안에서 마주 보고 있다는 느낌을 받을 수 있다. **Cup, plate, food, vase, camera object와 같은 familiar object**는 사용자가 실제 크기를 어느 정도 알고 있는 object이기 때문에, scene 안에서 거리와 크기를 판단하는 데 도움을 준다. 또한 menu board는 장면이 단순한 테이블 배치가 아니라 데이트 장소처럼 느껴지도록 만든다.

과제 예시에서 봄철 outdoor scene의 경우 cherry blossom tree를 배치할 수 있다고 설명되어 있었기 때문에, 본 구현에서는 **벚꽃나무와 꽃잎 particle**를 추가했다. 이를 통해 scene에 봄 분위기와 감성적 분위기를 부여했다. 꽃잎 particle은 정적인 scene에 작은 움직임을 추가하여, VR flirting scene이 더 생동감 있게 보이도록 한다.

공간감 측면에서는 occlusion과 motion parallax도 고려했다.

Occlusion은 가까운 object가 먼 object를 가리는 효과이다. 본 scene에서는 table, vase, dishes, camera object 등이 Avatar A의 시점과 Avatar B 사이에 배치되어 **partial occlusion(부분 가림)**을 만든다. 이로 인해 Avatar B가 배경 위에 단순히 떠 있는 캐릭터처럼 보이지 않고, table 너머에 실제로 서 있는 것처럼 보인다.

Motion parallax는 사용자의 시점이 움직일 때 가까운 object와 먼 object가 서로 다르게 움직이는 현상이다. 본 구현에서는 camera를 Avatar A의 head/neck 위치에 배치했기 때문에, 사용자는 Avatar A의 시점으로 scene을 볼 수 있다. 이때 가까운 table object와 먼 360도 video backdrop 사이에 상대적인 시각 변화가 발생하므로, 공간감이 강화된다.

본 단계에서 수행한 내용은 아래와 같다.

항목	수행 내용
Cherry blossom tree	scene 주변에 벚꽃나무를 배치하여 봄 분위기 형성
Petal particle	꽃잎 particle을 추가하여 움직임과 감성적 분위기 추가
Familiar object	cup, plate, food, vase, camera object 등을 배치하여 거리/크기 판단 cue 제공
Menu board	데이트 장소 같은 장소성 부여

Occlusion	table object가 Avatar A와 Avatar B 사이에 위치하도록 배치하여 partial occlusion(부분 가림) 효과 제공
Motion parallax	Avatar A 시점 camera를 통해 가까운 object와 먼 object의 상대적 움직임 제공



그림 1: 최종 HW4 VR flirting scene 구성 화면 (Girl(Avatar A)의 시점)

360도 video backdrop 안에 두 명의 avatar, table props, cherry blossom tree, petal particle, menu board, 사진 앨범 object가 함께 배치된 최종 scene

3. Avatar A/B 상호작용 구현

본 구현에서는 Girl avatar를 Avatar A로 설정했다. Avatar A는 사용자가 직접 조작하는 캐릭터이며, 사용자는 Avatar A의 시점에서 scene을 관찰한다. 이를 위해 camera를 Girl avatar의 neck/head 위치 근처에 배치했다. Game View에서는 **Avatar A의 코와 손 일부가 보이도록** 구성하였고, 이를 통해 사용자가 Avatar A의 몸 안에 들어가 있는 것 같은 self-body presence를 제공했다.

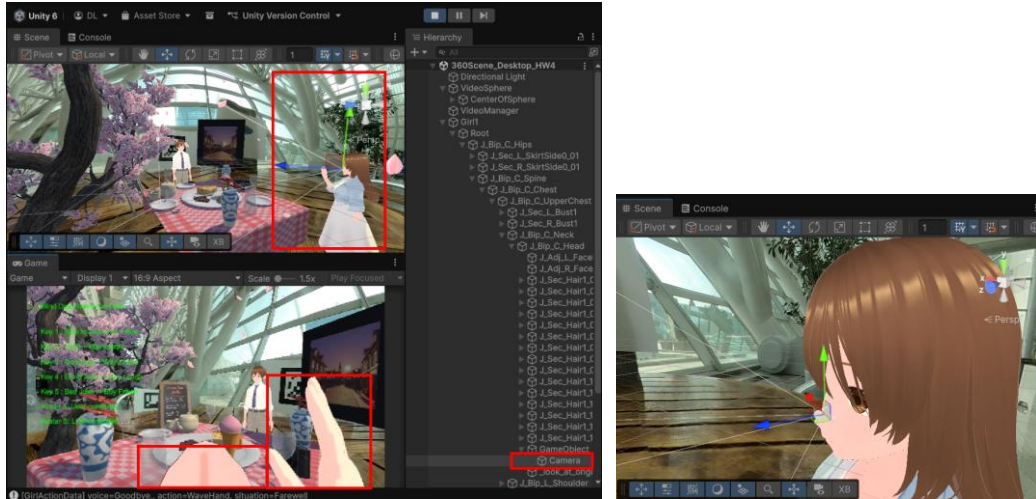


그림 2: Avatar A의 first-person viewpoint

Camera를 Girl Avatar의 head/neck 위치에 배치하여, 사용자가 Avatar A의 시점에서 Avatar B를 바라보도록 구성했다. Game View에서는 Avatar A의 코와 손 일부가 보이며, 이는 self-body presence를 강화한다.

Boy avatar는 Avatar B로 설정했다. Avatar A와 Avatar B는 table을 사이에 두고 서로 마주 보도록 배치했다. Avatar A의 시점에서 Avatar B가 정면에 보이도록 camera 위치와 avatar rotation을 조정했다. 이로 인해 사용자는 Avatar A가 되어 Avatar B와 대화하는 것처럼 scene을 경험할 수 있다.

Avatar A는 keyboard input으로 제어했다. 과제에서 keyboard, interaction controller, voice command 등이 가능한 입력 방식으로 제시되어 있었기 때문에, 본 구현에서는 안정적인 시연을 위해 keyboard input을 사용했다.

구현한 key input은 아래와 같다.

Key	Avatar A 행동	의미
1	Nice to meet you + Bow	첫 만남에 하는 격식 있는 인사
2	Hello + Wave hand	편한 첫 만남 인사
3	Goodbye + Wave hand	편한 작별 인사
4	Good joke + Talk (얼굴, 입 움직임)	좋은 농담
5	Bad joke + Talk (얼굴, 입 움직임)	어색한/이상한 농담

각 interaction에는 body animation과 facial expression뿐만 아니라, 상황에 맞는 voice clip도 함께 연결했다. 예를 들어 첫 만남 인사에서는 “Nice to meet you”에 해당하는 음성이 재생되고, hello/goodbye 상황에서는 각각의 “Hello”, “Goodbye”와 같은 인사 음성이 실행된다. 또한 good joke와 bad joke 상황에서는 Girl Avatar가 농담을 말하는 음성 clip을 재생한 뒤, Boy가 웃거나 싫어하는 반응 음성을 재생하도록 구성했다. 이 음성 clip들은 실시간 TTS

로 생성한 것이 아니라, AI 음성 생성 프로그램을 이용해 미리 제작한 뒤 Unity에 AudioClip 형태로 import한 것이다. 이후 각 상황에 맞게 AudioSource에서 해당 clip을 재생하도록 연결했다.

따라서 본 구현의 interaction은 단순한 키 입력 기반 animation 실행이 아니라, **body animation, facial expression, voice clip**이 함께 동작하는 **multimodal response** 구조이다. 특히 Avatar B의 경우 LLM이 voice, action, expression, emotion을 출력하고, Unity는 이 중 action과 expression을 animation 및 표정으로 실행하며, 해당 상황에 맞는 기존 voice clip을 함께 재생한다.

이처럼 각 key input은 Avatar A의 animation, expression, voice clip을 실행한다. 또한 같은 입력은 AI server로 보낼 **GirlActionData**로 변환된다.

GirlActionData는 Avatar A의 행동 정보를 구조화하기 위해 사용했다. 이 data class는 voice, action, situation 세 가지 정보를 가진다. 예를 들어 Key 4를 누르면 다음과 같은 data가 생성된다.

```
voice = "I told a funny joke."
```

```
action = "Talk"
```

```
situation = "GoodJoke"
```

이 정보는 Python AI server로 JSON 형태로 전송된다. 즉, Avatar A의 행동은 단순히 Unity 안에서 animation을 실행하는 데 그치지 않고, Avatar B의 AI 기반 반응을 생성하기 위한 입력 데이터로도 사용된다.

Avatar B의 반응은 **BoyResponseData**로 관리했다. **BoyResponseData**는 voice, action, expression, emotion 네 가지 요소로 구성된다. Unity는 이 값을 받아 Avatar B의 Animator trigger와 VRM facial expression을 실행한다.

예를 들어 GoodJoke 상황에서 AI server가 다음과 같은 response를 반환할 수 있다.

```
voice = "Haha, that was so funny!"
```

```
action = "Laugh"
```

```
expression = "HappyTalk"
```

```
emotion = "Positive"
```

Unity는 이를 받아 Boy avatar의 laugh animation과 happy/talking expression을 실행한다. 즉, LLM이 Avatar B를 직접 3D로 움직이는 것이 아니라, LLM이 Avatar B의 반응 data를 생성하고, Unity가 그 data를 실제 animation과 facial expression으로 변환하여 실행하는 구

조이다.

4. LLM 기반 AI 반응 시스템

본 과제의 핵심 확장점은 기존 scripted interaction을 LLM 기반 AI response system으로 바꾼 것이다. 기존 HW2-3에서는 Key4를 누르면 Boy가 항상 정해진 방식으로 웃고, Key5를 누르면 항상 정해진 방식으로 찡그리는 구조였다. HW4에서는 이를 확장하여, Unity가 Avatar A의 행동 정보를 Python server로 보내고, Python server가 open-source LLM을 이용해 Avatar B의 반응을 생성하도록 구현했다.

전체 구조는 아래와 같다.

Unity

→ *GirlActionData* 생성

→ HTTP POST로 Python server에 JSON 전송

→ Python server가 Ollama LLM 호출

→ LLM이 *BoyResponseData* 생성

→ Python server가 Unity로 JSON 반환

→ Unity가 Avatar B animation/expression 실행

Unity는 `http://127.0.0.1:8000/ai` 주소로 request를 보낸다. Python server는 이 요청을 받은 뒤, 내부에서 Ollama API인 `http://127.0.0.1:11434/api/generate`를 호출한다.

본 구현에서는 **Ollama**를 사용하여 local 환경에서 open-source LLM을 실행했다. 사용한 model은 llama3.2:3b이다. Windows PowerShell에서 `ollama pull llama3.2:3b` 명령으로 model을 준비하였고, Python server에서 해당 model을 호출하여 Avatar B의 response를 생성했다.

Python server의 `build_prompt()` 함수에서는 LLM에게 Avatar B의 역할과 persona를 설명한다. Avatar B의 persona는 **“VR blind date scene의 캐릭터이며, 친절하고 약간 수줍은 20대 초반 대학생”**으로 설정했다. 또한 prompt에는 Avatar A의 현재 행동 정보인 **voice, action, situation**이 포함된다. 따라서 LLM은 현재 상황이 첫 만남인지, 인사인지, 좋은 농담인지, 어색한 농담인지 파악한 뒤 Avatar B의 반응을 생성한다.

LLM의 output은 Unity에서 실행 가능한 값으로 제한했다. LLM에게는 반드시 하나의 JSON object만 반환하도록 지시하였고, 반환해야 하는 field는 아래 네 가지로 제한했다.

field	의미
voice	Avatar B의 대사
action	Avatar B의 body animation
expression	Avatar B의 facial expression

emotion	Avatar B의 감정 상태
---------	-----------------

허용된 **action** 값은 Bow, WaveHand, Talk, Laugh, Frown, Angry, Idle이다. 허용된 **expression** 값은 Happy, HappyTalk, Angry, Talk, Neutral이다. 허용된 **emotion** 값은 Positive, Negative, Neutral이다. 이렇게 제한한 이유는 Unity에서 실제로 실행할 수 있는 animation과 expression이 정해져 있기 때문이다. (하지만, 추후 시간적 여유를 가지고 다양한 **action**, **expression**, **emotion**을 만든 뒤, 다양한 선택지 중 하나를 선택하는 방식으로 확장한다면, 더욱 실감 나는 AI 기반 대화 방식을 구현할 수 있을 것이라 생각한다.)

LLM이 항상 완벽한 JSON을 반환하거나, 항상 Unity에서 실행 가능한 값을 반환한다고 보장할 수는 없다. 따라서 Python server에는 validation 과정이 포함되어 있다.

validate_boy_response() 함수는 LLM이 반환한 action, expression, emotion이 허용된 목록에 포함되는지 검사한다. 만약 허용되지 않은 값이 나오면 action은 Idle, expression은 Neutral 같은 **안전한 기본값으로 대체**한다.

또한 **fallback** 구조도 구현했다. LLM 호출이 실패하면 Python server는 rule_based_fallback()을 사용한다. 이 함수는 situation 값에 따라 미리 정의된 안정적인 response를 반환한다. 추가적으로 Python server 자체가 꺼져 있는 경우에는 Unity의 RequestBoyResponseFromServer()에서 network error가 발생하고, Unity 내부의 GenerateTemporaryAIResponse()를 호출한다. 따라서 Python server가 꺼져 있어도 Unity scene은 멈추지 않고 기존 rule-based 방식으로 계속 동작한다.

AI 기반 반응 시스템에서 수행한 내용은 아래와 같다.

항목	수행 내용
Python server	HTTP server 구현
Unity request	GirlActionData를 JSON으로 Python server에 전송
Ollama	llama3.2:3b model 사용
LLM prompt	Avatar B persona 및 Avatar A situation 포함
LLM output	voice/action/expression/emotion JSON 생성
Validation	허용되지 않은 값은 Idle/Neutral 등으로 보정
Python fallback	LLM 실패 시 rule-based response 사용
Unity fallback	Python server 실패 시 Unity 내부 임시 response 사용
Avatar B control	LLM response를 animation/expression으로 변환하여 실행

추가적으로, 단순 interaction 조건과 최종 구현 버전을 비교하여 VR UX의 차이를 관찰했다. 얼굴 표정을 추가하기 전에는 Avatar B가 몸 동작과 voice clip만으로 반응했기 때문에, 사용

자가 Avatar B의 감정 상태와 상호작용 맥락을 직관적으로 이해하기 어려운 부분이 있었다. 반면 최종 버전에서는 body animation, facial expression, voice clip이 함께 실행되도록 구성하였고, 그 결과 Avatar B의 반응이 훨씬 자연스럽게 느껴졌다. 특히 good joke와 bad joke 상황에서 표정이 함께 제시되면서 긍정적 반응과 어색한 반응의 차이가 더 명확하게 전달되었다. 이를 통해 얼굴 표정, 몸 동작, voice를 통합하는 것이 VR flirting scene의 social presence와 affective quality를 강화하는 데 효과적임을 확인하였다.

5. AR Marker 기반 오브젝트 배치

과제4에서는 360도 video에 포함된 marker를 이용해 object를 overlay하라고 요구했다. 이를 위해 본 구현에서는 360도 video에서 실제 AR marker를 검출하고, 검출된 marker 정보를 Unity에서 사용할 수 있는 위치 정보로 변환하여 사진 앨범 object를 overlay하여 배치했다.

Marker-based item placement의 전체 흐름은 다음과 같다.

360도 video frame

→ OpenCV ArUco marker detection

→ marker ID 및 marker center 추출

→ marker center를 yaw/pitch 각도로 변환

→ detection 결과를 CSV로 저장

→ Unity에서 CSV file load

→ yaw/pitch를 Unity direction vector로 변환

→ ARMarkerAnchor_Frame01 위치 계산

→ album cube를 marker anchor의 child로 배치

Marker detection은 Unity 내부에서 직접 수행하지 않고, Python과 OpenCV를 이용하여 수행했다. Python script는 360도 video를 frame 단위로 읽고, 각 frame에서 ArUco marker를 검출했다. 검출된 marker에 대해 `frame, time, marker_id, center_x, center_y, yaw, pitch` 정보를 CSV로 저장했다. 검출 결과에서 marker ID 17이 여러 번 나타났기 때문에, Unity에서는 marker ID 17을 target marker로 사용했다.

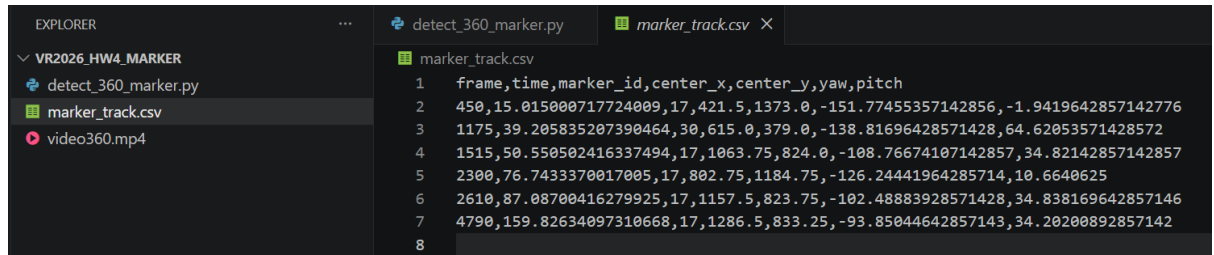


그림 3: CSV 데이터 추출 모습

360도 video는 equirectangular projection을 사용하므로, 2D pixel 좌표는 3D 방향과 대응된다. 따라서 marker center의 x 위치는 yaw, y 위치는 pitch로 변환했다. 사용한 변환식은 아래와 같다.

$$yaw = (center_x / video_width) \times 360 - 180$$

$$pitch = 90 - (center_y / video_height) \times 180$$

이 방식은 marker의 완전한 6-DoF pose를 계산하는 것은 아니지만, 360도 scene 내부에서 marker가 어느 방향에 있는지를 계산하는 데 충분했다.

Unity에서는 MarkerTrackCsvPlayer.cs script를 사용하여 CSV file을 읽었다. CSV는 Unity의 Resources folder에 배치하였고, script는 target marker ID에 해당하는 marker sample을 찾아 yaw/pitch 값을 읽었다. 이후 yaw/pitch 값을 3D direction vector로 변환하고, 아래와 같이 marker anchor의 위치를 계산했다.

$$markerAnchor.position = sceneCenter.position + direction \times markerDistance$$

여기서 sceneCenter는 360도 video sphere의 중심이고, markerDistance는 scene center로부터 marker anchor까지의 거리이다.

초기에는 video playback time에 따라 marker anchor가 계속 갱신되어 album cube가 흔들리거나 이동하는 문제가 있었다. 이를 해결하기 위해 최종 구현에서는 target marker ID의 첫 번째 valid sample을 선택하고, 해당 marker sample을 계속 사용하도록 했다. 즉, 최종 position을 고정한 것이 아니라 yaw/pitch 값을 가진 marker sample을 고정했다. 이 방식은 object를 안정적으로 유지하면서도, markerDistance, yawOffsetDegrees, pitchOffsetDegrees 등 Inspector parameter를 통해 placement를 계속 조정할 수 있다는 장점이 있다.

최종적으로 ARMarkerAnchor_Frame01 object를 생성하고, 그 child로 사진 앨범 object를

overlay하여 배치했다. Marker anchor는 detected yaw/pitch direction에 따라 배치되며, 사진 앨범 object는 marker anchor를 따라간다. 또한 marker anchor가 sphere center를 바라보도록 회전시켜, album cube가 scene 내부의 사용자 방향을 향하도록 했다.

본 구현의 핵심은 object를 단순히 눈대중으로 배치한 것이 아니라, OpenCV ArUco marker detection 결과를 기반으로 배치했다는 점이다.

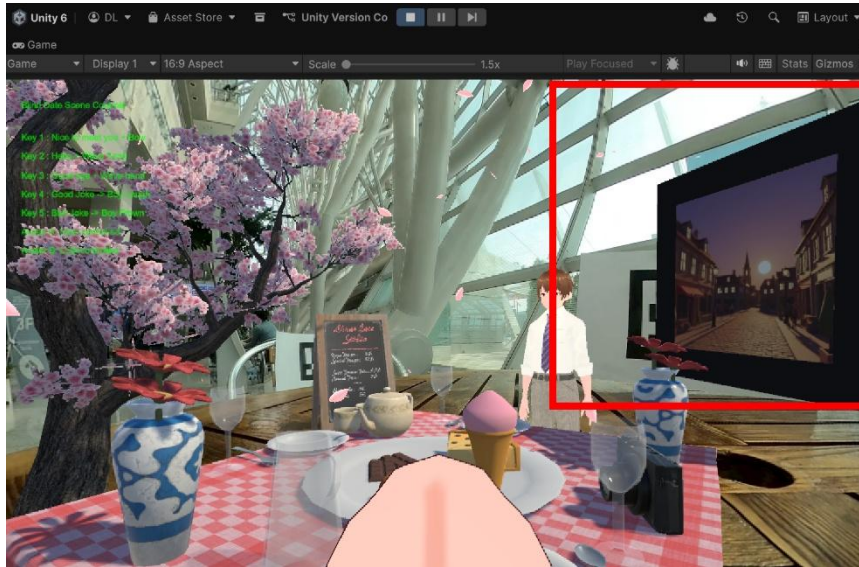


그림 4: AR marker 기반 사진 앨범 object 배치 결과

6. Windows Build

본 프로젝트는 Windows build 형태로 실행할 수 있다. 다만 LLM 기반 AI response는 Unity build 내부에 포함되어 있는 것이 아니라, 외부 Python server와 Ollama server를 통해 작동한다. 따라서 Windows build 형태에서 AI 기반 반응을 사용하려면 아래 세 가지가 필요하다.

Ollama가 설치되어 있어야 한다.

llama3.2:3b model이 준비되어 있어야 한다.

ai_server_llm.py Python server가 실행 중이어야 한다.

Windows build 실행 시 구조는 다음과 같다.

Windows executable

→ Python LLM server

→ *Ollama llama3.2:3b*

→ *Python server*

→ *Windows executable*

만약 Python server 또는 Ollama가 실행 중이지 않다면, Unity는 network error를 감지하고 내부 fallback response를 사용한다. 따라서 build 자체는 멈추지 않지만, 이 경우에는 LLM 기반 response가 아니라 rule-based response로 동작한다.

7. 결론

본 과제에서는 기존 VR blind date scene을 Assignment 4의 요구사항에 맞게 확장했다. 공간감 강화를 위해 벚꽃나무, 꽃잎 particle, table props, menu board, familiar object 등을 추가하였고, **marker 기반 augmented object 배치**를 통해 360도 video backdrop의 AR marker를 활용했다.

또한 Girl avatar를 Avatar A로 설정하고, camera를 Avatar A의 head/neck 위치에 배치하여 사용자가 Avatar A의 시점에서 scene을 경험하도록 구성했다. **Key 1~5 입력을 통해 Avatar A의 행동(얼굴 표정, 몸 움직임, 대화)을 제어**할 수 있도록 했고, 이 행동 정보를 GirlActionData로 변환하여 **AI server**로 전송했다.

Avatar B는 Boy avatar로 설정하였고, Python server와 **Ollama 기반 open-source LLM**을 이용하여 **Avatar B의 response**를 생성했다. LLM은 Avatar A의 voice, action, situation을 입력받아 Avatar B의 voice, action, expression, emotion을 JSON 형태로 생성한다. Unity는 이 response를 받아 Avatar B의 animation trigger와 VRM facial expression을 실행한다.

최종적으로 본 구현은 **사용자 제어 Avatar A, AI 제어 Avatar B, marker 기반 augmented object, spatial presence enhancement, LLM 기반 response generation**을 포함하는 **VR flirting scene**으로 완성되었다. 기존 HW2-3의 scripted interaction에서 더 나아가, Avatar B가 외부 AI server를 통해 상황과 persona에 맞는 반응을 생성하도록 확장한 것이 본 과제의 핵심 결과이다.